

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I P. KALYANI (192472199) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Kalyani

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario

A government uses AI-powered drones to deliver emergency medicines in flood-affected areas. The drone must find the safest and fastest route despite blocked roads, bad weather, and changing travel costs.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path with the smallest heuristic value (estimated distance to the goal).

Formula:

$$f(n) = h(n)$$

Strengths

- Finds a route quickly.
- Simple and easy to implement.
- Suitable when fast decisions are needed.

Weaknesses

- Ignores actual travel cost.
- May choose blocked or unsafe paths.
- Does not always find the shortest route.

ii. A Search Algorithm*

A* considers both the actual travel cost and the estimated distance.

Formula:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = Cost from start to current node
- **$h(n)$** = Estimated cost to the goal

Advantages

- Finds the optimal path.
- Avoids risky and expensive routes.
- Performs better in dynamic environments.

iii. Impact of Heuristic Accuracy

- **Accurate heuristic:** Both algorithms perform well.
- **Poor heuristic:** Greedy Search may choose wrong paths, while A* still finds a better route by considering travel cost.

iv. Effect of Dynamic Changes (Blocked Paths)

- **Greedy Search:** May need to restart when a path is blocked, causing delays.
- *A Search:** Updates the route using new information and finds a safer alternative.

v. Recommended Algorithm

A Search* is the best choice because it:

- Finds the safest and shortest route.
- Considers travel cost and heuristic together.
- Adapts better to changing road and weather conditions.
- Provides reliable performance during emergencies.

Conclusion

For emergency medicine delivery, A* is more suitable than Greedy Best-First Search. Although Greedy Search is faster, A* provides better accuracy, safety, and adaptability, making it the best algorithm for real-time disaster response.

2. Scenario

A smart city uses an AI system to optimize traffic signal timings at many intersections. The goal is to reduce waiting time, fuel consumption, and traffic congestion. Since traffic changes continuously, the system must find better signal timings in a very large search space.

i. Formulate the Problem & Why Traditional Search is Inefficient

This is an optimization problem where the objective is to find the best traffic signal timings.

Objective:

- Minimize waiting time
- Minimize fuel consumption
- Reduce traffic congestion

Traditional search methods are inefficient because:

- The search space is very large.
- Traffic conditions change frequently.
- They take too much time to find the best solution.

ii. Hill Climbing Algorithm

Hill Climbing starts with an initial solution and continuously moves to a better neighboring solution until no further improvement is possible.

Limitations

- Can get stuck at a local maximum.
- Cannot always find the global optimum.
- Performs poorly in complex traffic conditions.

iii. Local Maxima, Plateaus, and Ridges

- **Local Maximum:** The algorithm finds a good solution but misses a better one.
- **Plateau:** Many neighboring solutions have the same value, so progress stops.
- **Ridge:** The best path requires several moves, making it difficult for Hill Climbing to continue.

In traffic systems, these problems may result in unnecessary congestion and longer waiting times.

iv. Simulated Annealing & Genetic Algorithms

Simulated Annealing

- Sometimes accepts worse solutions to escape local maxima.
- Has a better chance of finding the global optimum.

Genetic Algorithm

- Works with multiple solutions at the same time.
- Uses selection, crossover, and mutation to generate better traffic signal timings.
- Suitable for large and dynamic optimization problems.

v. Recommended Approach

Genetic Algorithm (GA) is the best choice because it:

- Handles large search spaces efficiently.
- Avoids getting stuck in local optima.
- Adapts to changing traffic conditions.
- Produces high-quality solutions for real-time traffic optimization.

Conclusion

For smart traffic signal optimization, Genetic Algorithm is more effective than Hill Climbing because it provides better scalability, adaptability, and optimization in dynamic traffic environments. It helps reduce congestion, waiting time, and fuel consumption more efficiently.

3. Scenario

An autonomous rover is deployed on Mars to explore unknown terrain. It must collect samples, avoid hazards, work with limited sensors, and make decisions without a complete map.

i. Why Online Search is Required

The rover does not have complete information about the environment before starting. It learns about the terrain while moving, so it needs an online search agent to make decisions in real time.

ii. Challenges of Partial Observability and Dynamic Environment

- The rover cannot see the entire terrain.
- Obstacles like rocks and craters may appear unexpectedly.

- Limited sensor range makes navigation difficult.
- The rover must continuously adapt to new information.

iii. Building and Updating the Internal Model

The rover:

- Collects data using sensors.
- Detects obstacles and safe paths.
- Updates its internal map as it explores.
- Uses the updated information to plan its next move safely.

iv. Online Search vs. Uninformed and Informed Search

Online Search	Uninformed Search	Informed Search
Works without a complete map	Needs complete information	Uses heuristic information
Learns while exploring	Searches blindly	Finds better paths using heuristics
Best for unknown environments	Less efficient	Efficient when the map is known

v. Recommended Approach

Online Search is the most suitable approach because it:

- Adapts to unknown environments.
- Makes decisions in real time.
- Updates its knowledge continuously.
- Improves safety by avoiding hazards.

Conclusion

For a Mars rover, Online Search is the best approach because the environment is unknown and constantly changing. It allows the rover to learn, adapt, and navigate safely while completing its exploration mission.

4. Scenario

A university wants to generate an exam timetable automatically. The timetable must avoid exam clashes, manage limited exam halls, consider faculty availability, and provide special accommodations for some students.

i. Formulate the Problem as a CSP

Variables

- Exam subjects

Domains

- Available time slots and exam halls

Constraints

- No student should have overlapping exams.
- Exam hall capacity should not be exceeded.
- Faculty must be available.
- Some exams must be scheduled before others.
- Special accommodation requirements must be satisfied.

ii. Backtracking Search

Backtracking assigns a time slot to each exam one by one. If a conflict occurs, it goes back to the previous assignment and tries another option until a valid timetable is found.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid time slots after each assignment.

Advantages

- Detects conflicts early.
- Reduces unnecessary searching.
- Improves the speed of timetable generation.

iv. Real-World Challenges and Best Strategy

Challenges

- Large number of courses and students.
- Limited exam halls.

- Faculty scheduling conflicts.
- Last-minute changes.

Best Strategy

Use Backtracking with Forward Checking because it efficiently handles constraints, reduces search time, and generates a valid timetable even for large universities.

Conclusion

Modeling exam scheduling as a Constraint Satisfaction Problem (CSP) helps create conflict-free timetables. Backtracking with Forward Checking is the best approach as it improves efficiency, handles multiple constraints, and scales well for real-world university scheduling.

5. Scenario

A gaming company is developing an AI opponent for a real-time strategy game. The AI must make quick decisions, evaluate many possible moves, and respond within a limited time.

i. Minimax Algorithm

The Minimax algorithm helps the AI choose the best move by assuming that the opponent will also play optimally.

- **MAX player:** Tries to maximize the score.
- **MIN player:** Tries to minimize the score.

The AI selects the move that gives the best possible outcome.

ii. Computational Challenges

- The game tree contains a very large number of possible moves.
- Evaluating every move requires high time and memory.
- Real-time games need faster decision-making.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by removing branches that cannot affect the final decision.

Advantages

- Reduces the number of nodes evaluated.
- Saves time and memory.
- Produces the same optimal result as Minimax but much faster.

iv. Evaluation Function

The evaluation function scores each game state based on factors such as:

- Number of available resources.
- Strength of army or units.
- Control of important areas.
- Health and position of units.

A higher score represents a better position for the AI.

v. Recommended Strategy

Minimax with Alpha-Beta Pruning is the best approach because it:

- Makes intelligent decisions.
- Reduces computation time.
- Works efficiently in large game trees.
- Balances optimality with real-time performance.

Conclusion

For real-time strategy games, Minimax with Alpha-Beta Pruning is the most suitable algorithm. It allows the AI to make fast, accurate, and strategic decisions while reducing unnecessary computations, making it ideal for complex game environments.